

SML Assignment 3

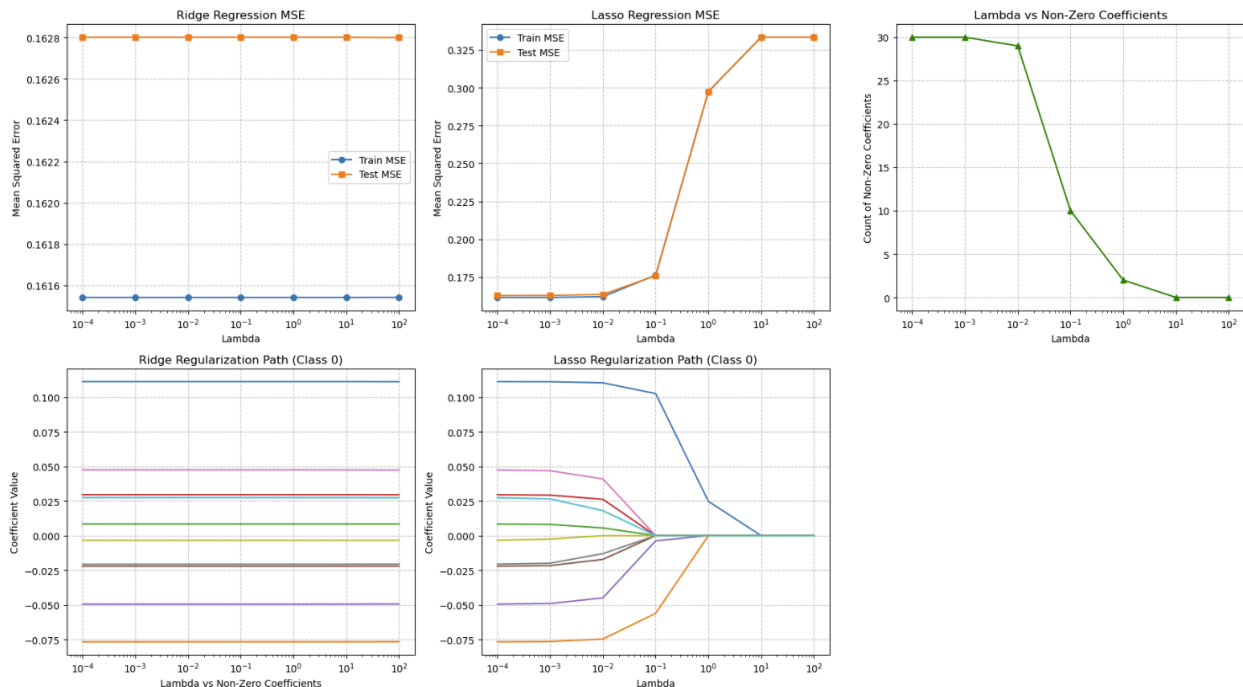
Mayank Yadav | 2024343

Question 1

Code & Approach Explanation

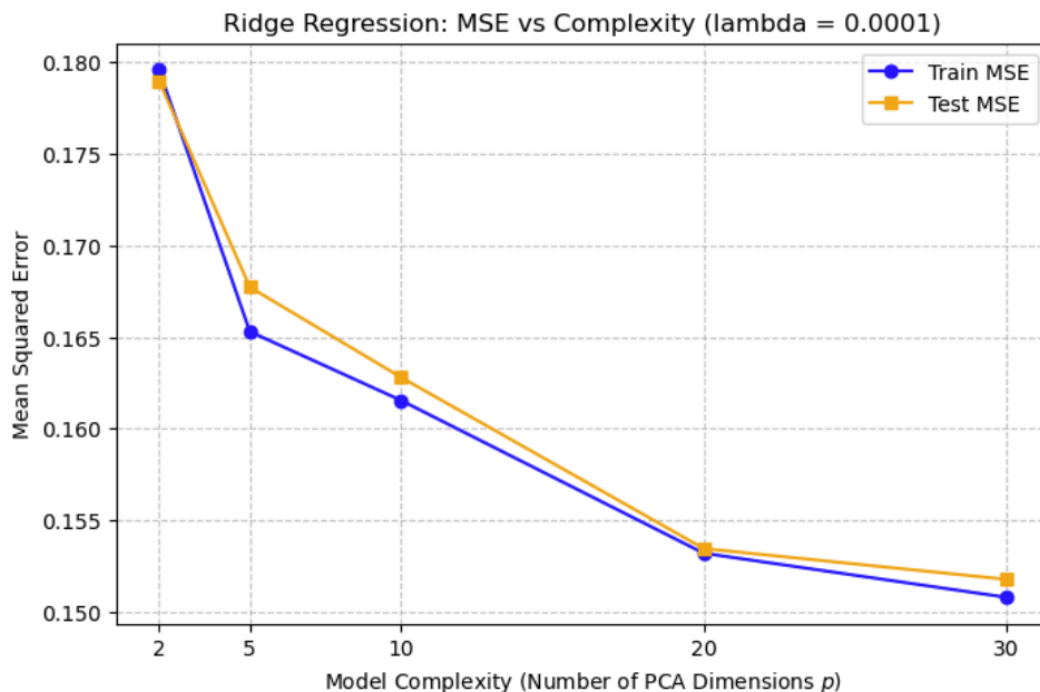
(Code is well documented with comments)

1. Preprocessing is similar to other assignments.
2. Use PCA code from assignment 2 to reduce the dataset to 10 features.
3. Implemented Ridge Regression by using numpy to implement the closed form solution for Ridge Regression.
4. Used scikit-learn for implementing Lasso Regression.
5. Performed ridge and lasso regression for the given lambda values, and plotted the obtained mse errors on train set and test set.
6. Lowest mse error was obtained for $\lambda = 10^{-4}$ or 0.0001 (may not be very evident from plots, but from printed values in the attached notebook, there is a small difference between all values making the **mse corresponding to $\lambda = 0.0001$ the lowest** for both train and test sets, see appendix)
7. Used $\lambda = 0.0001$ to perform pca for the given range of components to find the one with the **lowest test MSE, which is 30 components**.
8. Using $\lambda = 0.0001$ and 30 components for pca, the **final classification accuracy** obtained for both ridge and lasso regression models was **96.37% on the test data** and **96.50% on the train data**.



Plot Explanation

1. Observed increasing MSE with increasing lambda for Ridge Regression.
2. Observed increasing MSE with increasing lambda for Lasso Regression.
3. With increasing lambda and increasing MSE, the number of non-zero components for lasso regression fall as zero valued weights compensate for the large MSE.
4. Regularization paths show that ridge regression moves smoothly towards zero (slope not properly visible here due to values being extremely close, this is because for ridge regression, the MSE of predictions dominates due to large train set, which can be weighted by $1/N$ to observe a more expected graph) as lambda increases and mse increases. (ridge regression penalty on weights creates a circle around the origin)
5. Regularization paths show that lasso regression moves abruptly towards zero as mse increases (due to increase in lambda). (piecewise linear) (lasso regression penalty forms a quadrilateral around the origin)



Using the lambda corresponding to the lowest test MSE, the MSE for test and train sets is lowest for pca using 30 components. This is expected as more components in pca preserve higher variance, usually allowing for better classification.

Question 2

Code & Approach Explanation

(Code is well documented with comments)

1. Preprocessing and PCA recycled from previous question.

2. Implemented functions to calculate gini(S) and weighted gini index for a split using the formulae given in the assignment.
3. Trained a decision tree with three leafs by first finding the best split (lowest weighted gini index) for the root node by examining all splits.
4. I used the **median value of a feature for the split**, as it is more **robust to outliers** as compared to the mean value and ensures a 50-50 split.
5. Repeated the same strategy to find appropriate splits for the left and right subtrees, and finally selected the child split which allowed for minimum weighted gini index at the node.
6. Traversed the tree (as a dictionary) to find the prediction node, with prediction being the mode of the labels of the train samples going to that node.
7. Implemented bootstrapping by sampling with replacement from the original train dataset, and trained 5 trees from 5 bootstrapped datasets.
8. Found out of bag samples for each tree, and computed the oob-error for that tree by calculating the MSE for the predicted label using that tree and the actual label, and found the final average oob-error by averaging over all the trees. **(0.22 for bagging model, 0.39 for random forests)**
9. For random forests, change the function for finding the best split to see any 3 features randomly.
10. I used **k=3**, since PCA had 10 components and 3 is closest to square root of 10 which is a common value to take for k (as discussed in class), as it allows for enough decorrelation by randomizing the selected feature as it is not too high. Also improves the chance of finding the actual best split as it is not too low.
11. Predictions for forest models were done by taking majority votes, breaking ties as numpy breaks them.

Accuracy of Single Tree, Bagging Forest and Random Forest

1. The class-wise and overall classification accuracy was similar for a single tree **(76.23%)** and bagging implementation **(76.19%)**.
2. Accuracy was much higher for random forests implementation **(83.66%)**.
3. This is expected because random forests enforce decorrelation between trees by randomly picking features to split on and points taken in the dataset, reducing the overall expected variance and overfitting. Bagging may not be so effective if few features perform very good splits for all bootstrapped sets as then all trees split on the same few features, increasing correlation and overfitting due to high variance.
4. Class-wise accuracy for single tree, bagging and random forest respectively:
Single Tree

Class-wise Accuracy:
Class 0: 74.69187911531318%
Class 1: 99.74784930287748%
Class 2: 53.44075193017791%

TEST SET ACCURACY
Overall Classification Accuracy: 76.23133142675564%

Class-wise Accuracy:
Class 0: 74.08163265306122%
Class 1: 99.73568281938327%
Class 2: 52.42248062015504%

Bagging

TRAIN SET ACCURACY
Overall Classification Accuracy: 77.09821188852494%

Class-wise Accuracy:
Class 0: 74.96201249366875%
Class 1: 99.74784930287748%
Class 2: 53.5918093319906%

TEST SET ACCURACY
Overall Classification Accuracy: 76.19955513187162%

Class-wise Accuracy:
Class 0: 74.28571428571429%
Class 1: 99.64757709251101%
Class 2: 52.22868217054264%

Random Forest

TRAIN SET ACCURACY
Overall Classification Accuracy: 84.73393116039306%

Class-wise Accuracy:
Class 0: 82.69458044909675%
Class 1: 99.09522396914862%
Class 2: 70.51023833501175%

TEST SET ACCURACY
Overall Classification Accuracy: 83.66698442961551%

Class-wise Accuracy:
Class 0: 81.53061224489797%
Class 1: 99.20704845814979%
Class 2: 68.6046511627907%

Question 3

Code & Approach Explanation

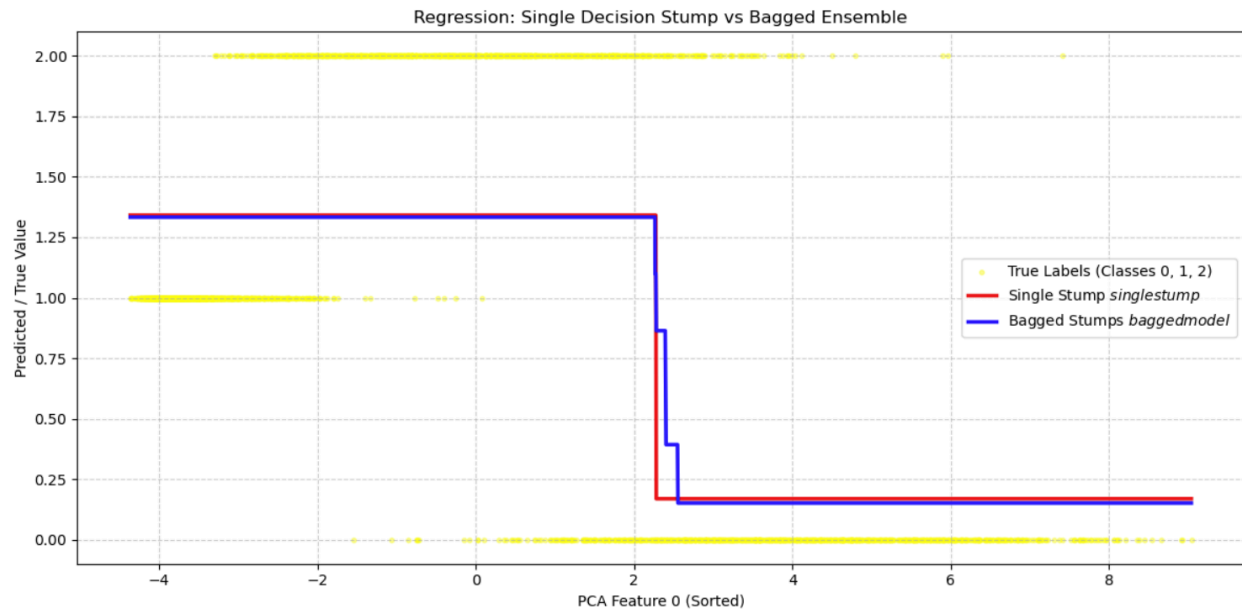
(code is well documented with comments)

1. Preprocessing and PCA recycled from previous question (only the dataset changed).

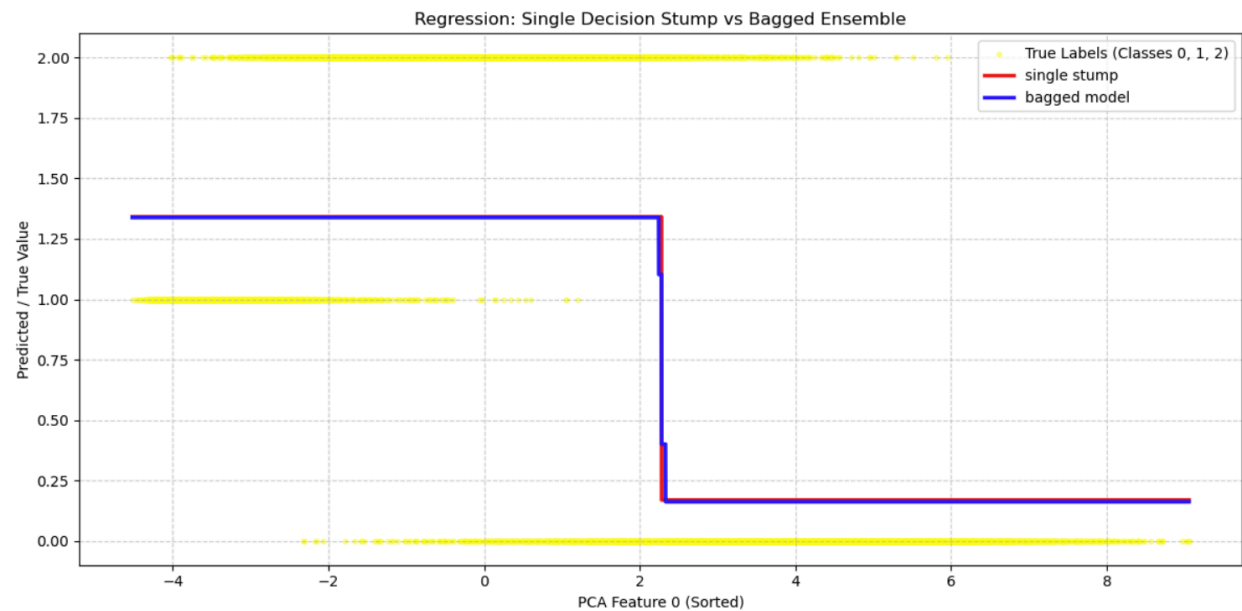
2. Implemented function to calculate SSR using the formula given in the assignment.
3. Implemented a single stump decision tree by performing splits along the mid points of all distinct points in sorted order, and computing the SSR for each split.
4. The final split was the one which minimized SSR, **MSE for single stump was 0.354**.
5. For the bagging forest, created bootstrapped datasets by sampling with replacement, and created a single stump regression tree for all bootstrapped datasets.
6. Found average out of bag error similar to question 2 (**average oob-error for bagging was 0.36**).
7. Averaged the predictions from all bootstrapped models to get final predictions for calculating test set MSE (**0.352**).
8. Plotted the predictions of single stump and bagging model predictions along with real labels for test set and train set data points.

Plot Explanation

Average Out-of-Bag (OOB) MSE: 0.3600
Test Set MSE (Bagging): 0.3521383559603276



Average Out-of-Bag (OOB) MSE: 0.3592
Test Set MSE (Bagging): 0.3544353137214656



The single stump regression tree splits the dataset into 2 parts, predicting the average for each part, effectively mapping every train set point and incoming test point to one of two regions, leading to a comparatively poorer classification than the bagging model, which due to 5 decision stumps, is able to create more regions by averaging the predicted values from each stump.

Single stump predictions look like a step function, while bagging model predictions looks like piece-wise step functions (staircase like structure).

The single stump has poorer classification as it underfits the feature space (high bias since only 2 regions), while the bagging model fits the feature space better than the single stump, as averaging over the predicted values reduces expected variance, and there are several regions allowing for more nuance and less underfitting as compared to the 2 region case.

Appendix

Test Set MSE for Lambda values

```
{0.0001: {'train_ridge': 0.16154236944285172,
'test_ridge': 0.1628007966709392,
'train_lasso': 0.16154241950646256,
'test_lasso': 0.16280304812584656,
'lasso_non_zero': 30},
0.001: {'train_ridge': 0.16154236944285183,
'test_ridge': 0.16280079665464292,
'train_lasso': 0.1615474620997794,
'test_lasso': 0.16283065911388778,
'lasso_non_zero': 30},
0.01: {'train_ridge': 0.1615423694428549,
'test_ridge': 0.1628007964916819,
'train_lasso': 0.16202805775847232,
'test_lasso': 0.16351934868575624,
'lasso_non_zero': 29},
0.1: {'train_ridge': 0.16154236944306974,
'test_ridge': 0.1628007948622502,
'train_lasso': 0.17623341756395233,
'test_lasso': 0.17591357692582377,
'lasso_non_zero': 10},
1: {'train_ridge': 0.16154236946364597,
'test_ridge': 0.16280077858579248,
'train_lasso': 0.29739285316894304,
'test_lasso': 0.29751167944794105,
'lasso_non_zero': 2},
10: {'train_ridge': 0.16154237151142478,
'test_ridge': 0.16280061760641631,
'train_lasso': 0.3333333333333333,
'test_lasso': 0.3333333333333333,
'lasso_non_zero': 0},
100: {'train_ridge': 0.16154257541931913,
'test_ridge': 0.16279918561099815,
'train_lasso': 0.3333333333333333,
'test_lasso': 0.3333333333333333,
'lasso_non_zero': 0}}
```